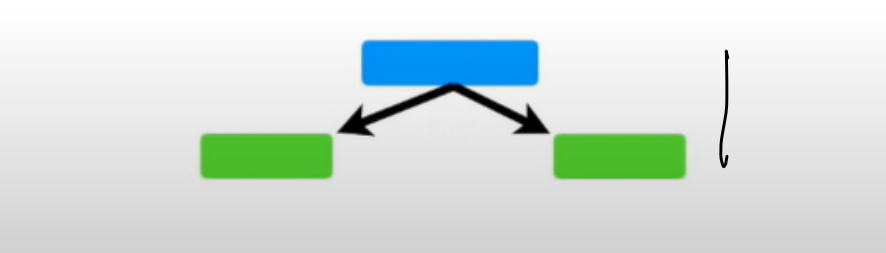
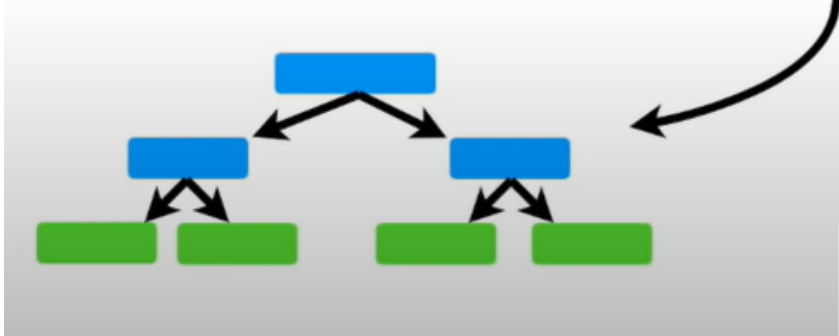
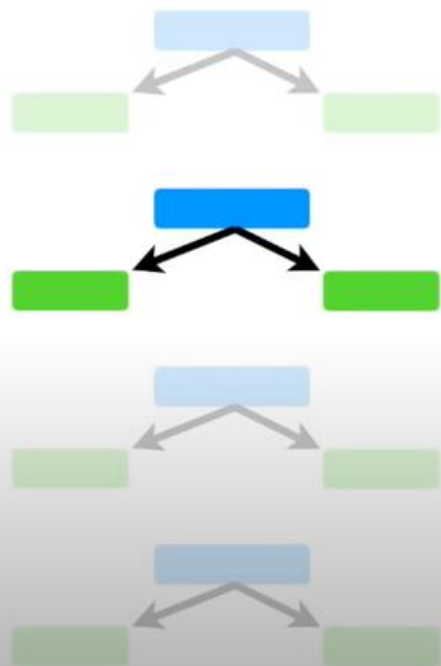


AdaBoost

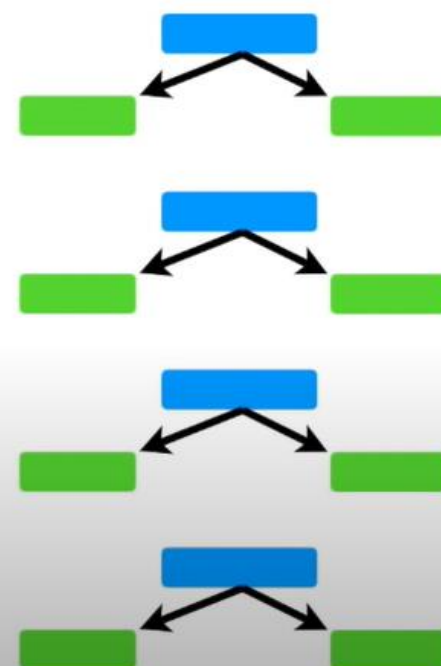
= Adaptive Boosting



A tree with just one node and two leaves is called a ***stump***.



...so this is really a **Forest of Stumps** rather than trees.



For example, if we were using this data to determine if someone had heart disease or not...



Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	No	Yes	167	Yes

...but a **Stump** can only use one variable to make a decision.

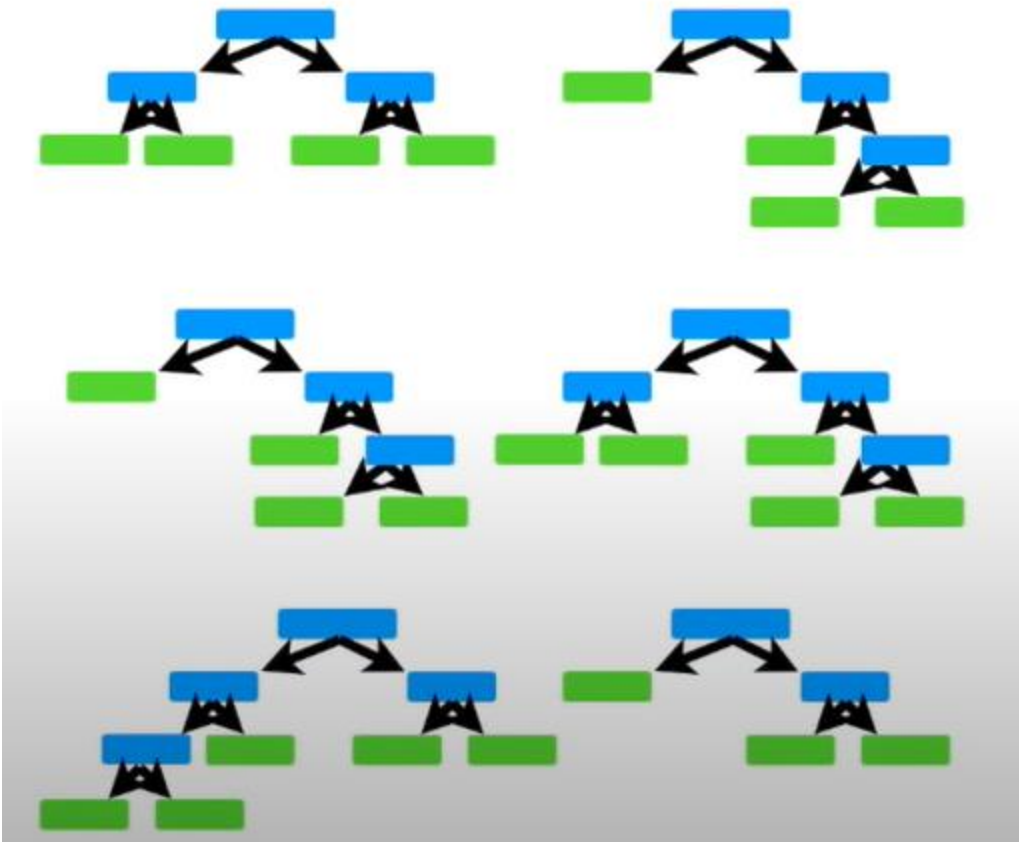
Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	No	Yes	167	Yes



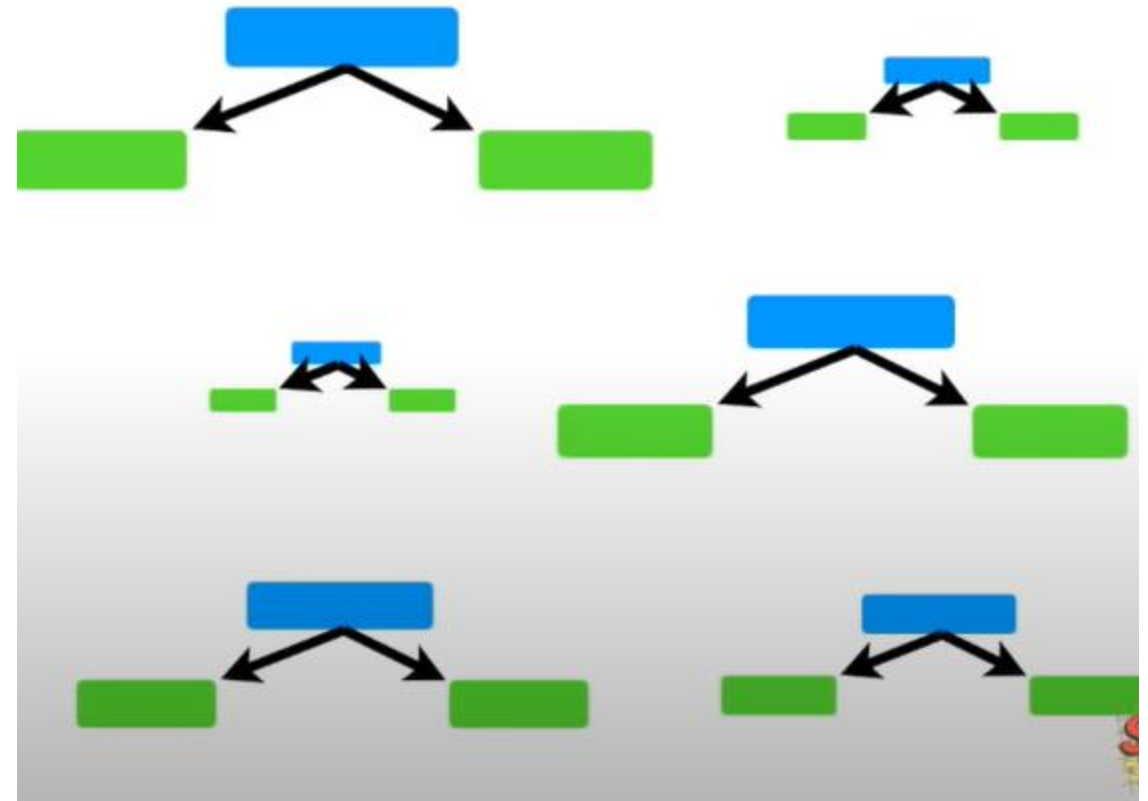
Thus, **Stumps** are technically "weak learners".

However, that's the way **AdaBoost** likes it, and it's one of the reasons why they are so commonly combined.

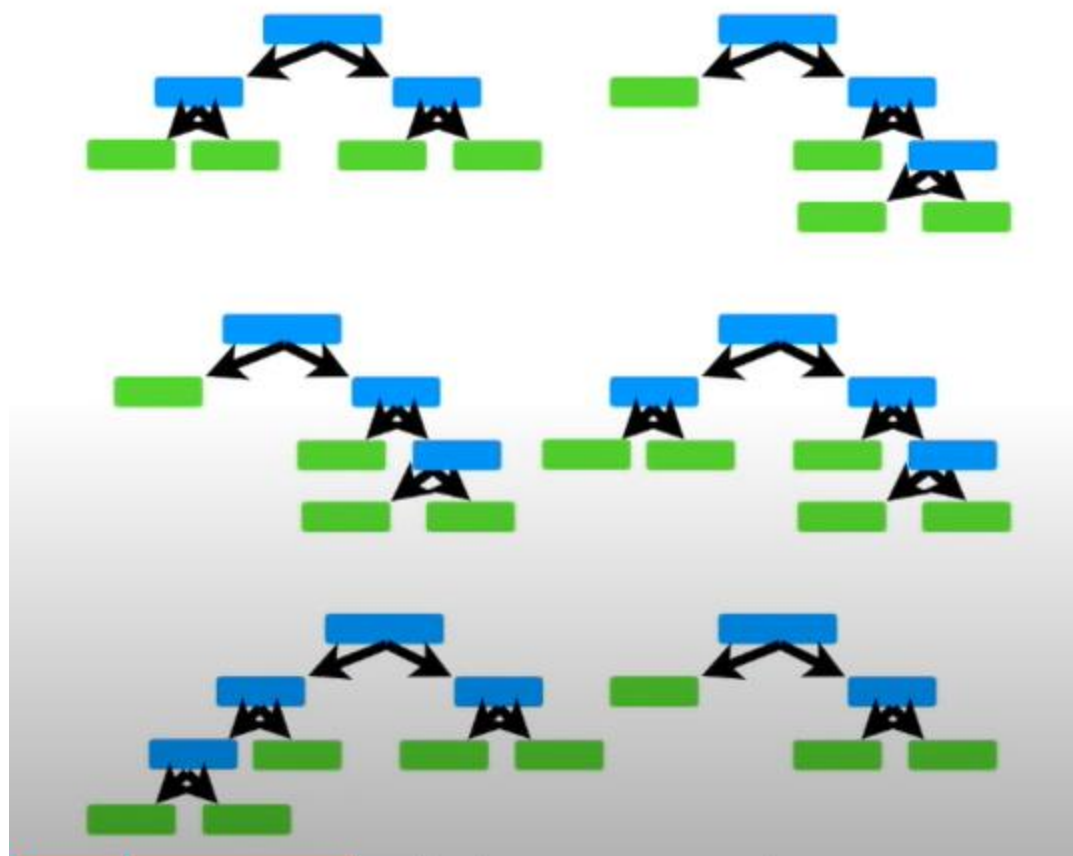
In a **Random Forest**, each tree has an equal vote on the final classification.



In contrast, in a **Forest of Stumps** made with **AdaBoost**, some stumps get more say in the final classification than others.



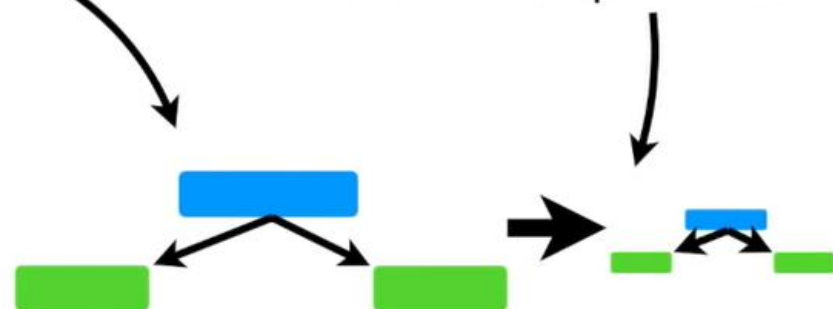
Lastly, in a **Random Forest**, each decision tree is made independently of the others.



In contrast, in a **Forest of Stumps** made with **AdaBoost**, order is important.

The errors that the first stump makes...

...influence how the second stump is made...



Now let's dive into the nitty gritty detail of how to create a **Forest of stumps** using AdaBoost

Chest Pain	Good Blood Circ.	Blocked Arteries	Weight	Heart Disease
No	No	No	125	No
Yes	Yes	Yes	180	Yes
Yes	Yes	No	210	No
Yes	No	Yes	167	Yes

Sample weight

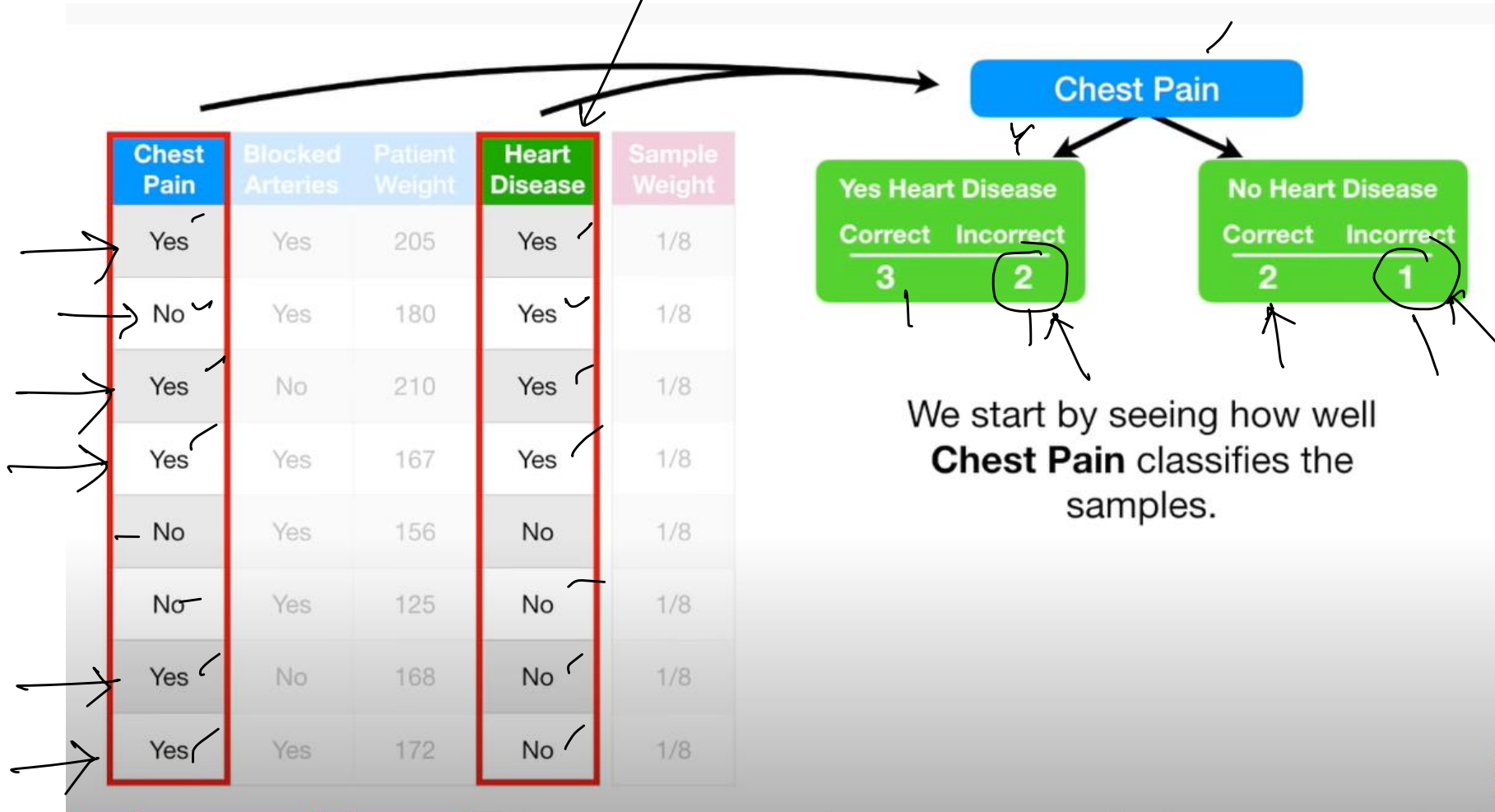
Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight
Yes	Yes	205	Yes	1/8
No	Yes	180	Yes	1/8
Yes	No	210	Yes	1/8
Yes	Yes	167	Yes	1/8
No	Yes	156	No	1/8
No	Yes	125	No	1/8
Yes	No	168	No	1/8
Yes	Yes	172	No	1/8

At the start, all samples get the same weight...

$$\frac{1}{\text{total number of samples}} = \frac{1}{8}$$

...and that makes the samples all equally important.

Decide weak learner

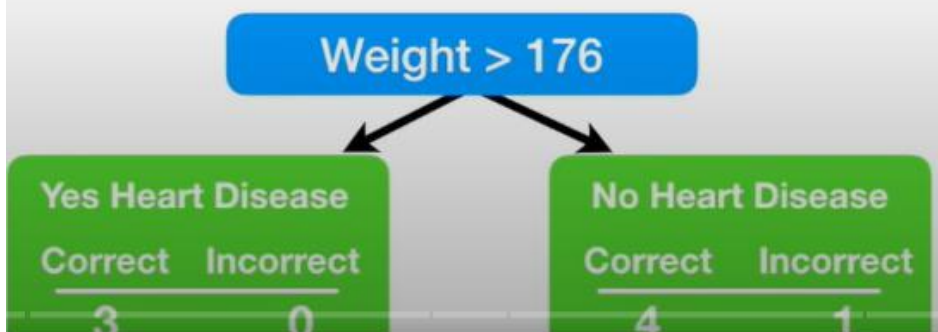


Similarly for

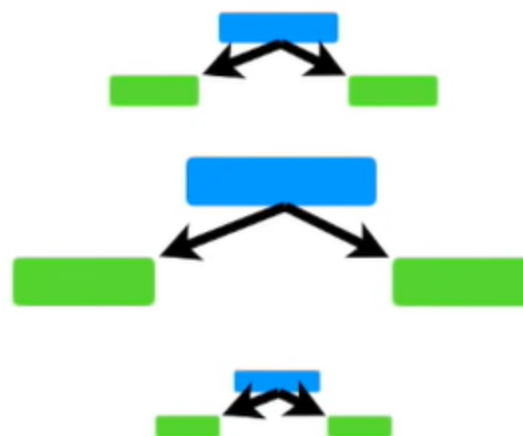
- **Blocked arteries**
- **Patient weight**

“Amount of say” by a weak learner

Now we need to determine how much say this stump will have in the final classification.



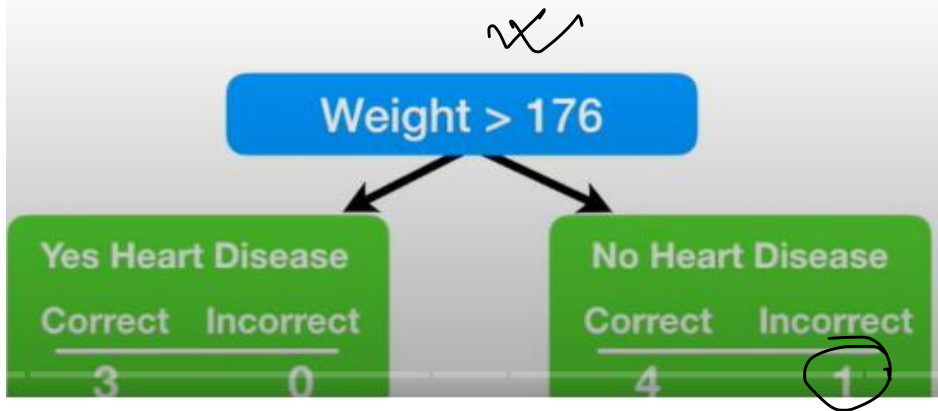
We determine how much say a stump has in the final classification based on how well it classified the samples.



Remember, some stumps get more say in the final classification than others.

The **Total Error** for a stump is the sum of the weights associated with the *incorrectly* classified samples.

Now we need to determine how much say this stump will have in the final classification.



Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight
Yes	Yes	205	Yes	1/8
No	Yes	180	Yes	1/8
Yes	No	210	Yes	1/8
Yes	Yes	167	Yes	1/8
No	Yes	156	No	1/8
No	Yes	125	No	1/8
Yes	No	168	No	1/8
Yes	Yes	172	No	1/8

Thus, in this case, the **Total Error** is $1/8$.

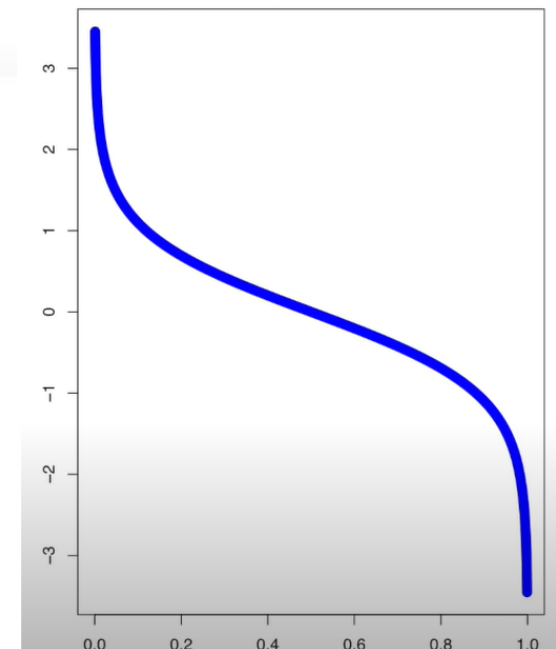


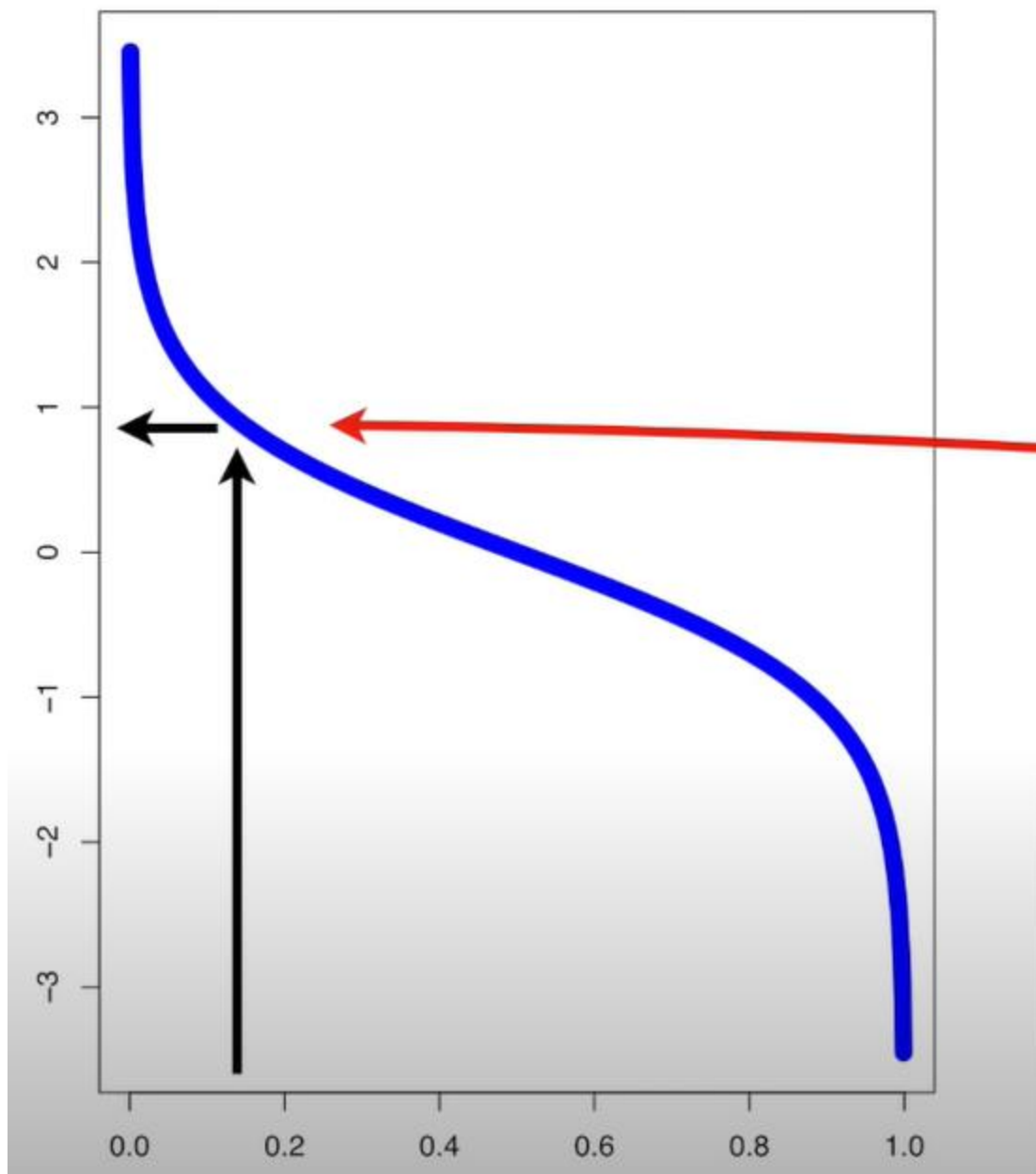
Because all the sample weights add up to 1, the **total error** will be between 0 (perfect classifier) and 1 (horrible classifier)

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight
Yes	Yes	205	Yes	1/8
No	Yes	180	Yes	1/8
Yes	No	210	Yes	1/8
Yes	Yes	167	Yes	1/8
No	Yes	156	No	1/8
No	Yes	125	No	1/8
Yes	No	168	No	1/8
Yes	Yes	172	No	1/8

We use the **Total Error** to determine **Amount of Say** this stump has in the final classification with the following formula:

$$\text{Amount of Say} = \frac{1}{2} \log\left(\frac{1 - \text{Total Error}}{\text{Total Error}}\right)$$





In our example, the error is $1/8$

...and the **Amount of Say** that this stump has on the final classification is **0.97**.

$$\text{Amount of Say} = \frac{1}{2} \log(7) = 0.97$$

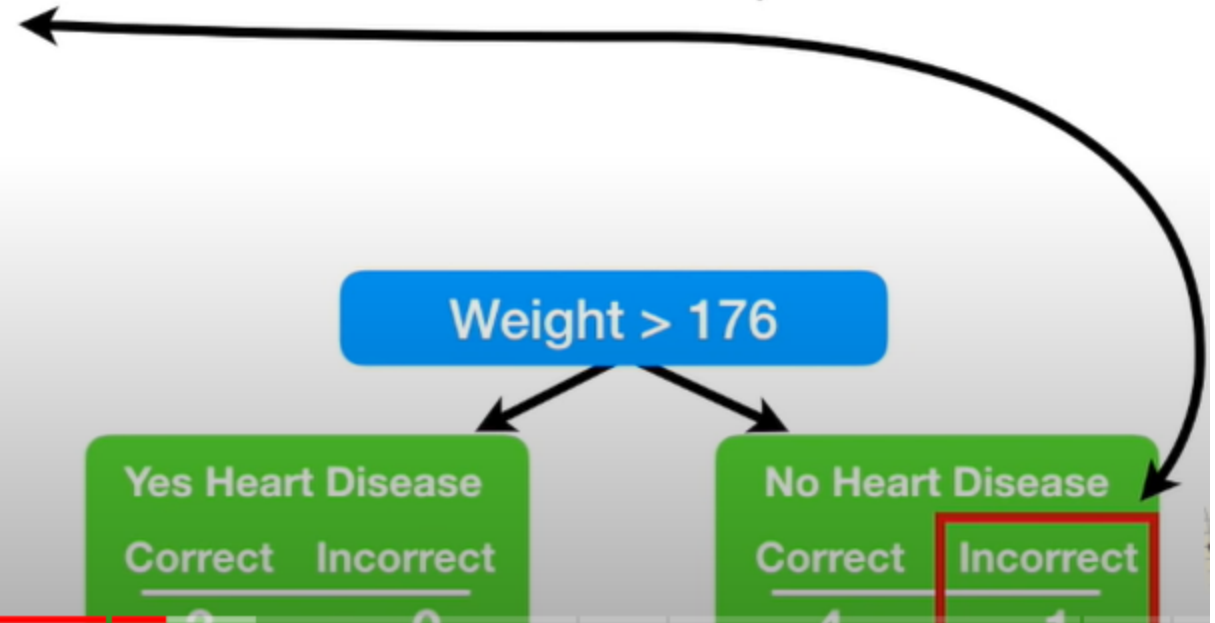
Now we need to learn how to
modify the weights so that the
next stump will take the errors that
the current stump made into
account.



...we will emphasize the need for the next stump to correctly classify it by increasing its **Sample Weight**...

... since this stump *incorrectly* classified this sample...

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight
Yes	Yes	205	Yes	1/8
No	Yes	180	Yes	1/8
Yes	No	210	Yes	1/8
Yes	Yes	167	Yes	1/8
No	Yes	156	No	1/8
No	Yes	125	No	1/8
Yes	No	168	No	1/8
Yes	Yes	172	No	1/8



Increase the sample weight of misclassification

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight
Yes	Yes	205	Yes	1/8
No	Yes	180	Yes	1/8
Yes	No	210	Yes	1/8
Yes	Yes	167	Yes	1/8
No	Yes	156	No	1/8
No	Yes	125	No	1/8
Yes	No	168	No	1/8

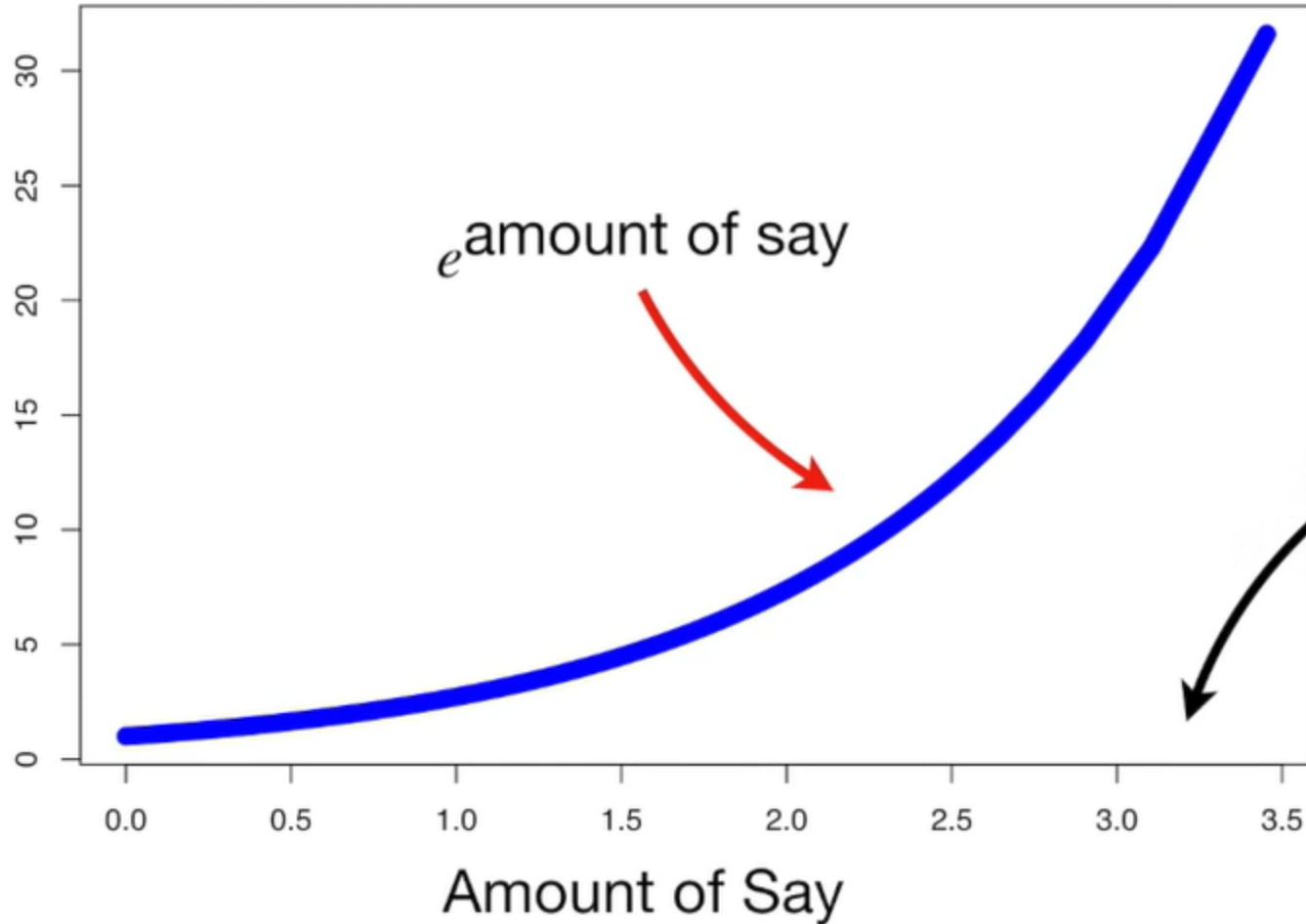
New Sample Weight = sample weight $\times e^{\text{amount of say}}$



This is the formula we will use to *increase* the **Sample Weight** for the sample that was *incorrectly* classified.

New Sample Weight = sample weight $\times e^{\text{amount of say}}$

$$= \frac{1}{8} e^{\text{amount of say}}$$



When the **Amount of Say** is relatively large, (i.e. the last stump did a good job classifying samples)...

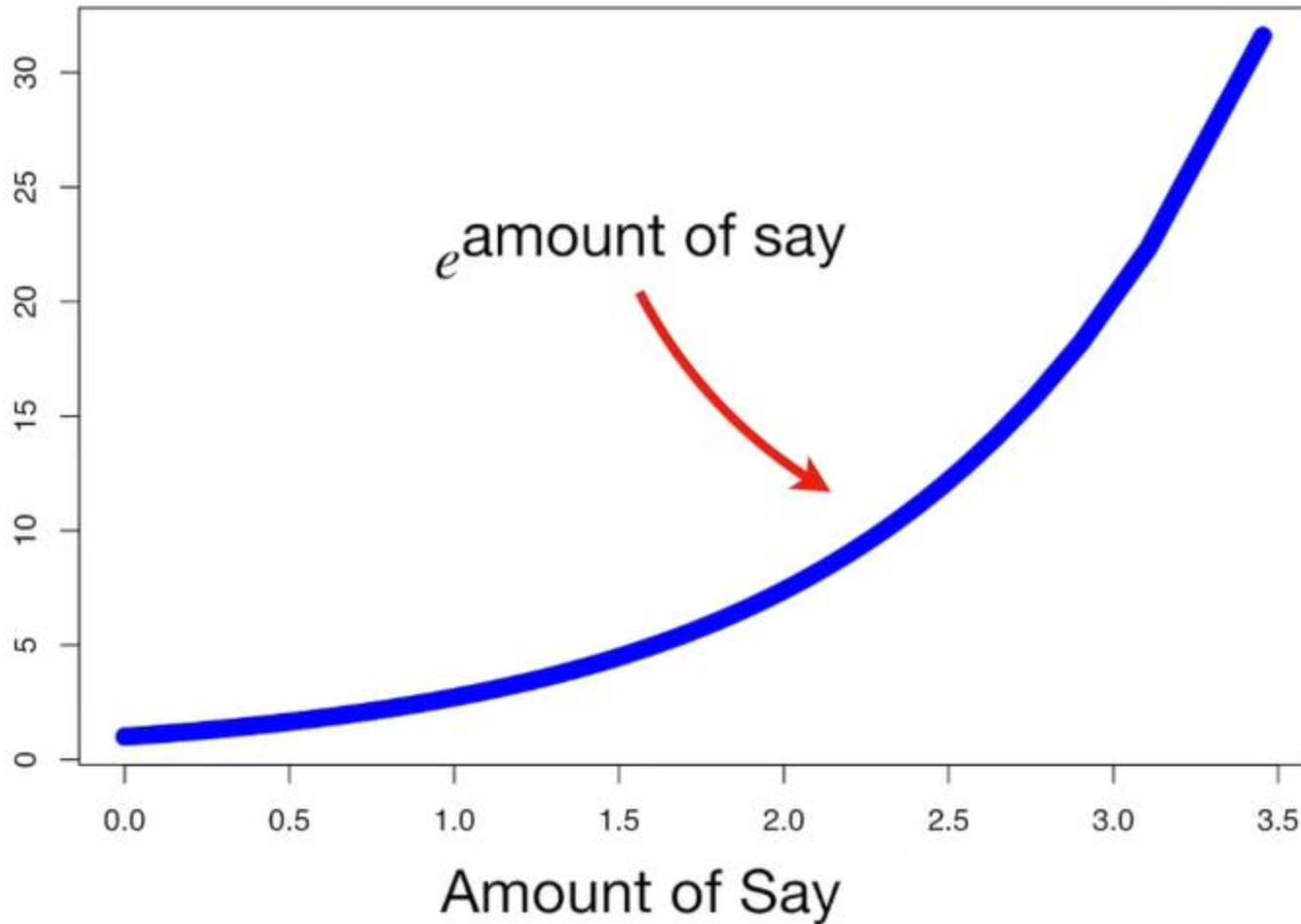
...then we will scale the previous **Sample Weight** with a large number.

This means that the **New Sample Weight** will be much larger than the old one.

New Sample Weight = sample weight $\times e^{\text{amount of say}}$

$$= \frac{1}{8} e^{\text{amount of say}}$$

$$= \frac{1}{8} e^{0.97} = \frac{1}{8} \times 2.64 = 0.33$$



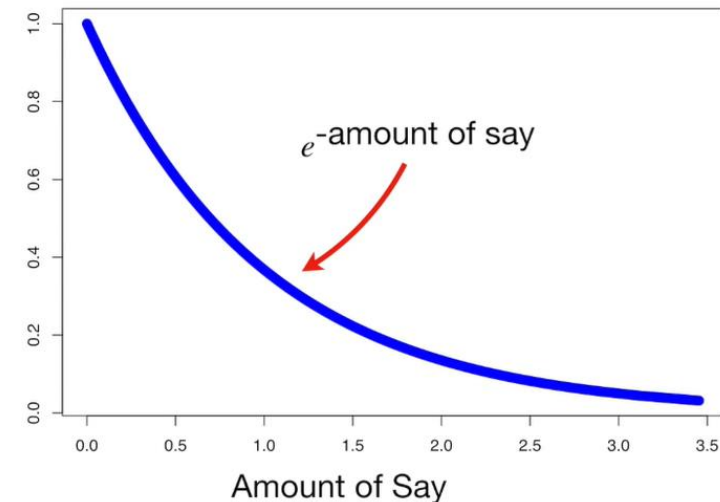
That means the new **Sample Weight** is **0.33**, which is *more* than the old one ($1/8 = 0.125$).

Decrease the weight of correctly classified instances

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight
Yes	Yes	205	Yes	1/8
No	Yes	180	Yes	1/8
Yes	No	210	Yes	1/8
Yes	Yes	167	Yes	1/8
No	Yes	156	No	1/8
No	Yes	125	No	1/8
Yes	No	168	No	1/8
Yes	Yes	172	No	1/8

New Sample Weight = sample weight $\times e^{-\text{amount of say}}$

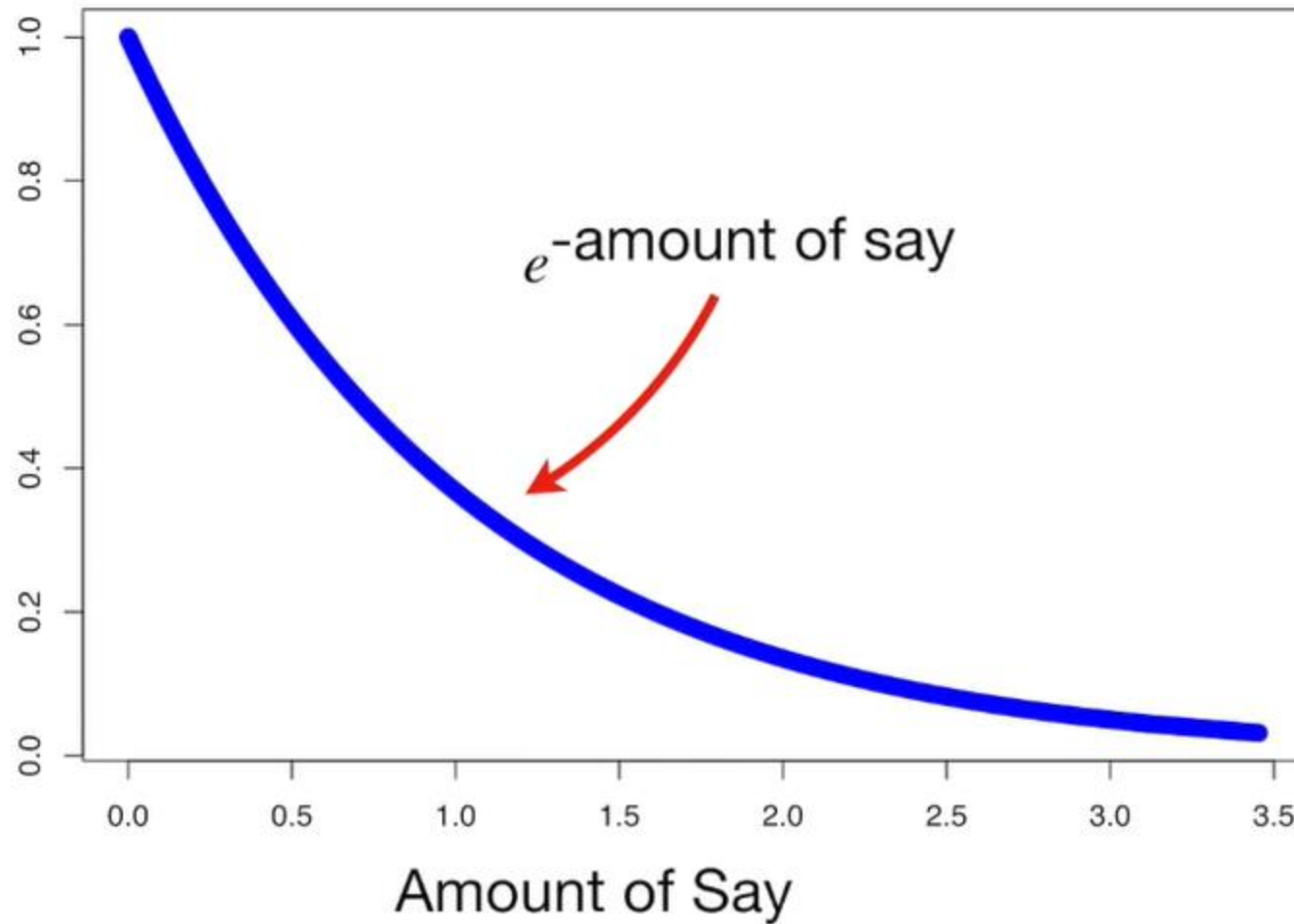
This is the formula we will use to *decrease* the **Sample Weights**.



New Sample Weight = sample weight $\times e^{-\text{amount of say}}$

$$= \frac{1}{8} e^{-\text{amount of say}}$$

$$= \frac{1}{8} e^{-0.97} = \frac{1}{8} \times 0.38 = 0.05$$



The new **Sample Weight** is **0.05**, which is *less* than the old one ($1/8 = 0.125$).

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight	New Weight
Yes	Yes	205	Yes	1/8	
No	Yes	180	Yes	1/8	
Yes	No	210	Yes	1/8	
Yes	Yes	167	Yes	1/8	0.33
No	Yes	156	No	1/8	
No	Yes	125	No	1/8	
Yes	No	168	No	1/8	
Yes	Yes	172	No	1/8	

We plug in **0.33** for the sample that was *incorrectly* classified...

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight	New Weight
Yes	Yes	205	Yes	1/8	0.05
No	Yes	180	Yes	1/8	0.05
Yes	No	210	Yes	1/8	0.05
Yes	Yes	167	Yes	1/8	0.33
No	Yes	156	No	1/8	0.05
No	Yes	125	No	1/8	0.05
Yes	No	168	No	1/8	0.05
Yes	Yes	172	No	1/8	0.05

All of the other samples get **0.05**.

Normalize the sample weights

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight	New Weight
Yes	Yes	205	Yes	1/8	0.05
No	Yes	180	Yes	1/8	0.05
Yes	No	210	Yes	1/8	0.05
Yes	Yes	167	Yes	1/8	0.33
No	Yes	156	No	1/8	0.05
No	Yes	125	No	1/8	0.05
Yes	No	168	No	1/8	0.05
Yes	Yes	172	No	1/8	0.05

Now we need to normalize the **New Sample Weights** so that they will add up to 1.

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight	New Weight	Norm. Weight
Yes	Yes	205	Yes	1/8	0.05	0.07
No	Yes	180	Yes	1/8	0.05	0.07
Yes	No	210	Yes	1/8	0.05	0.07
Yes	Yes	167	Yes	1/8	0.33	0.49
No	Yes	156	No	1/8	0.05	0.07
No	Yes	125	No	1/8	0.05	0.07
Yes	No	168	No	1/8	0.05	0.07
Yes	Yes	172	No	1/8	0.05	0.07

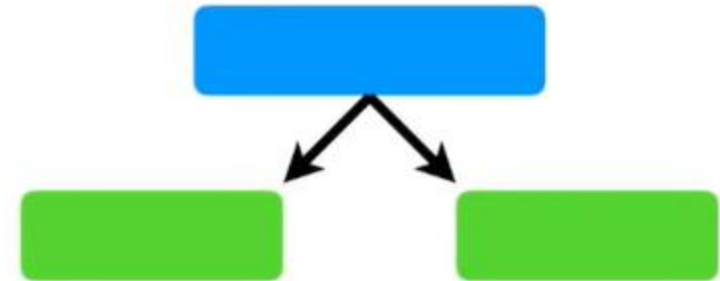
Sum=0.68

So we divide each **New Sample Weight** by **0.68** to get the normalized values.

Selecting the next weak learner

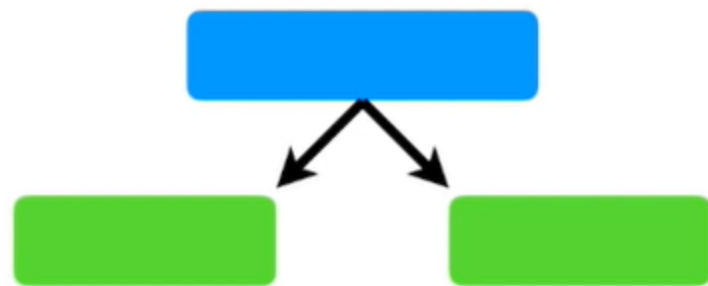
Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight
Yes	Yes	205	Yes	0.07
No	Yes	180	Yes	0.07
Yes	No	210	Yes	0.07
Yes	Yes	167	Yes	0.49
No	Yes	156	No	0.07
No	Yes	125	No	0.07
Yes	No	168	No	0.07
Yes	Yes	172	No	0.07

Now we can use the modified **Sample Weights** to make the second **stump** in the forest.



Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight
Yes	Yes	205	Yes	0.07
No	Yes	180	Yes	0.07
Yes	No	210	Yes	0.07
Yes	Yes	167	Yes	0.49
No	Yes	156	No	0.07
No	Yes	125	No	0.07
Yes	No	168	No	0.07
Yes	Yes	172	No	0.07

In theory, we could use the **Sample Weights** to calculate **Weighted Gini Indexes** to determine which variable should split the next stump.



Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight
Yes	Yes	205	Yes	0.07
No	Yes	180	Yes	0.07
Yes	No	210	Yes	0.07
Yes	Yes	167	Yes	0.49
No	Yes	156	No	0.07
No	Yes	125	No	0.07
Yes	No	168	No	0.07
Yes	Yes	172	No	0.07



Alternatively, instead of using a **Weighted Gini Index**, we can make a new collection of samples that contains duplicate copies of the samples with the largest **Sample Weights**.

How?
Roulette wheel

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight
Yes	Yes	205	Yes	0.07
No	Yes	180	Yes	0.07

We then continue to pick random numbers and add samples to the new collection until we the new collection is the same size as the original.

Yes				
Yes				
No				
No	Yes	125	No	0.07
Yes	No	168	No	0.07
Yes	Yes	172	No	0.07

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease
No	Yes	156	No
Yes	Yes	167	Yes
No	Yes	125	No
Yes	Yes	167	Yes
Yes	Yes	167	Yes
Yes	Yes	172	No
Yes	Yes	205	Yes
Yes	Yes	167	Yes

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight
Yes	Yes	205	Yes	0.07
No	Yes	180	Yes	0.07
Yes	No	210	Yes	0.07
Yes	Yes	167	Yes	0.49
No	Yes	156	No	0.07
No	Yes	125	No	0.07
Yes	No	168	No	0.07
Yes	Yes	172	No	0.07

Ultimately, this sample was added to the new collection of samples **4** times, reflecting its larger **Sample Weight**.

Chest Pain	Blocked Arteries	Patient Weight	Heart Disease
No	Yes	156	No
Yes	Yes	167	Yes
No	Yes	125	No
Yes	Yes	167	Yes
Yes	Yes	167	Yes
Yes	Yes	172	No
Yes	Yes	205	Yes
Yes	Yes	167	Yes



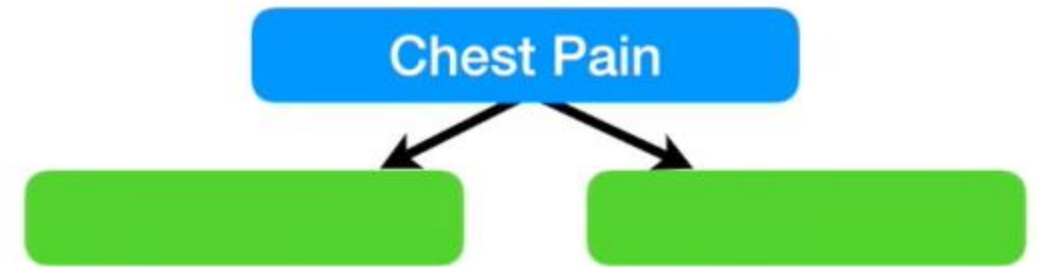
Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight
No	Yes	156	No	1/8
Yes	Yes	167	Yes	1/8
No	Yes	125	No	1/8
Yes	Yes	167	Yes	1/8
Yes	Yes	167	Yes	1/8
Yes	Yes	172	No	1/8
Yes	Yes	205	Yes	1/8
Yes	Yes	167	Yes	1/8



Lastly, we give all the samples equal **Sample Weights**, just like before.

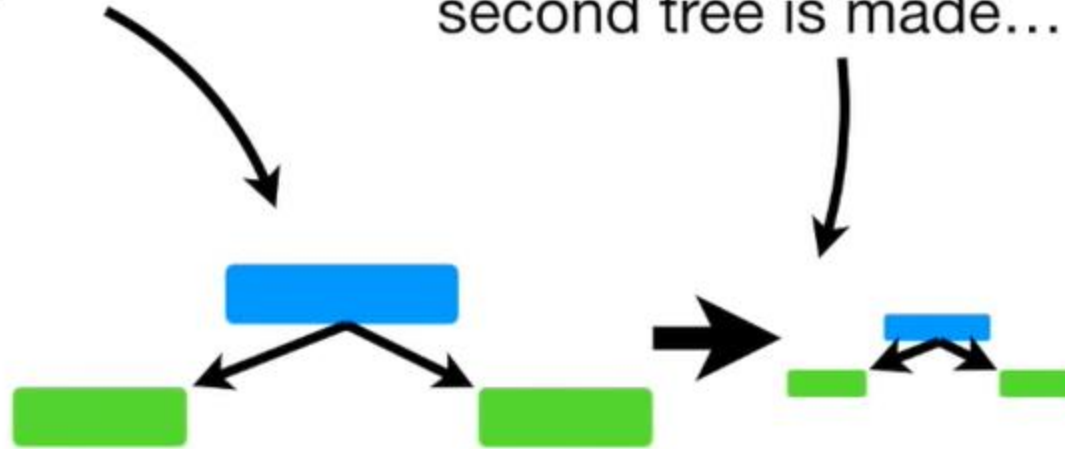
Chest Pain	Blocked Arteries	Patient Weight	Heart Disease	Sample Weight
No	Yes	156	No	1/8
Yes	Yes	167 ✓	Yes	1/8
No	Yes	125	No	1/8
Yes	Yes	167 ✓	Yes	1/8
Yes	Yes	167 ✓	Yes	1/8
Yes	Yes	172	No	1/8
Yes	Yes	205	Yes	1/8
Yes	Yes	167 ✓	Yes	1/8

Now we go back to the beginning and try to find the stump that does the best job classifying the new collection of samples.

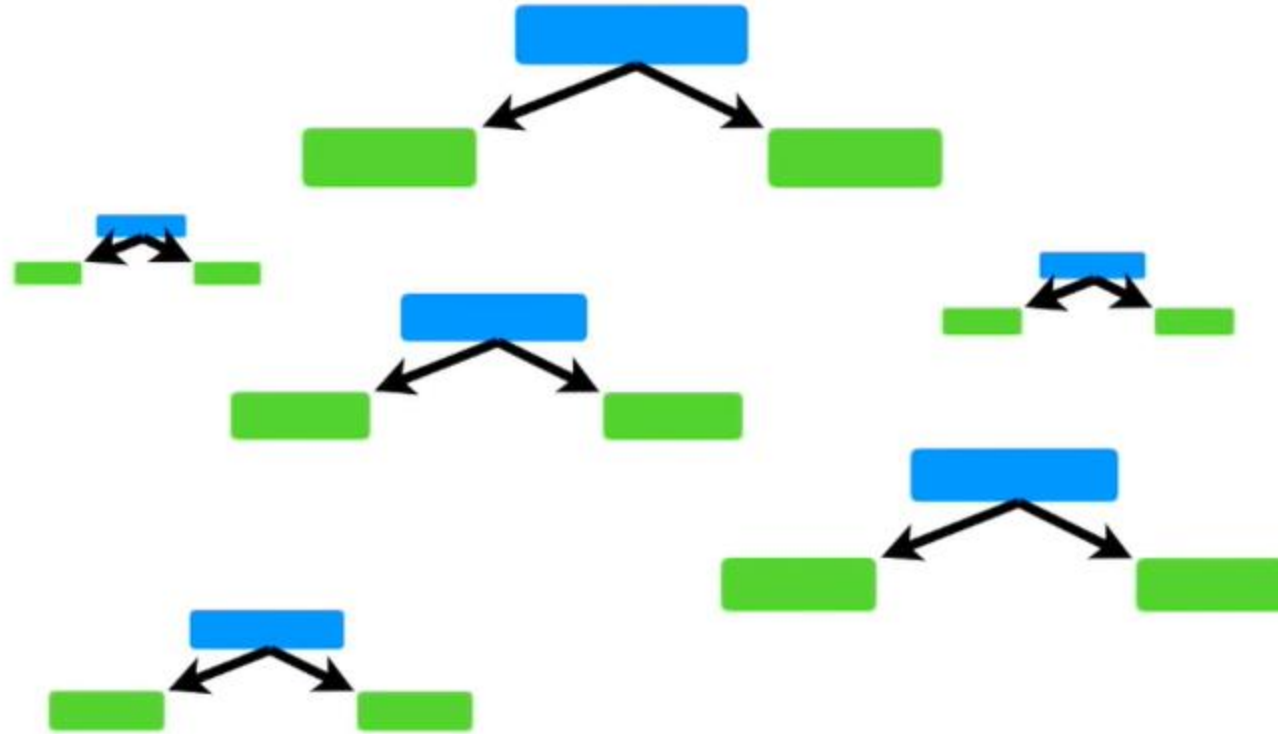


So that is how the errors
that the first tree
makes...

...influence how the
second tree is made...

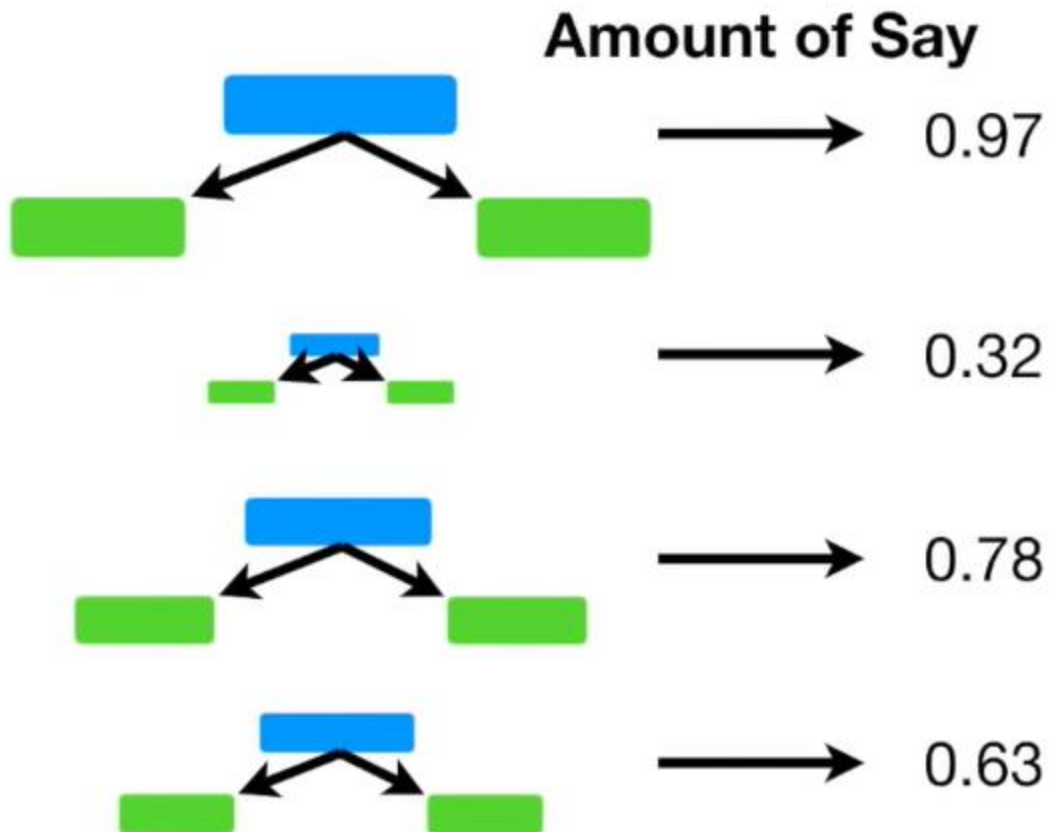


Now we need to talk about how a forest of stumps created by **AdaBoost** makes classifications...



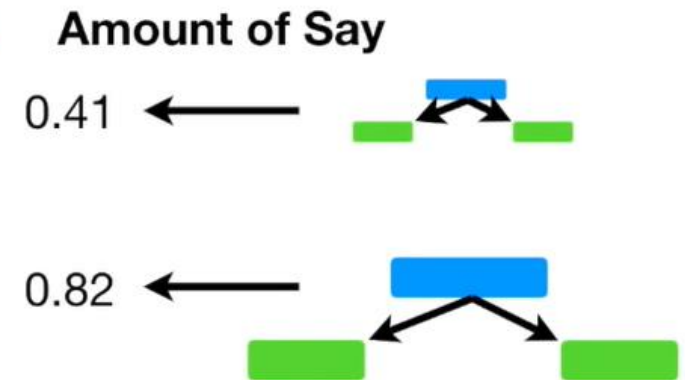
These are the **Amounts of Say** for these stumps...

Has Heart Disease



...and these are the **Amounts of Say** for these stumps...

Does Not Have Heart Disease



Ultimately, the patient is classified as **Has Heart Disease** because this is the larger sum.

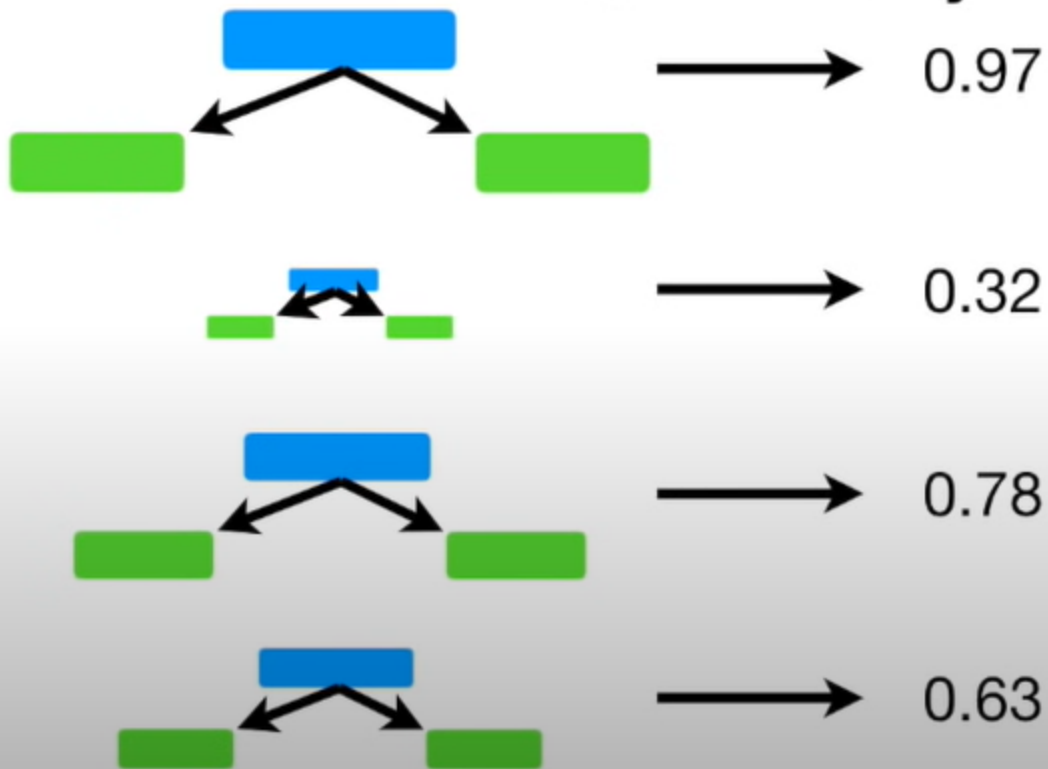
Has Heart Disease

Total = 2.7

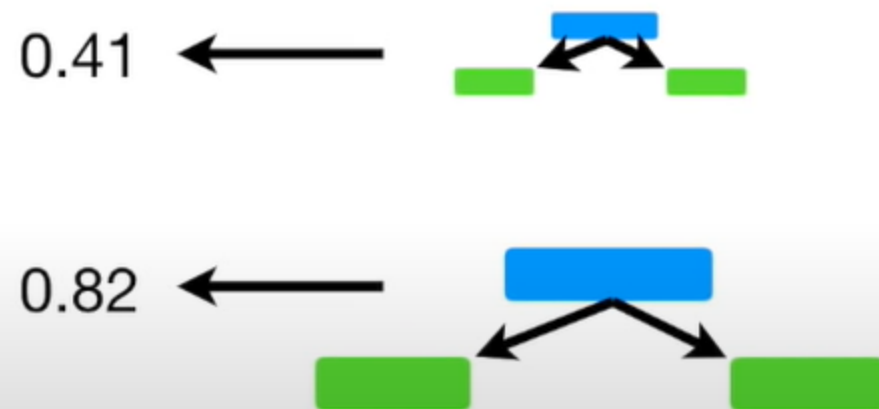
Total = 1.23

Does Not Have Heart Disease

Amount of Say

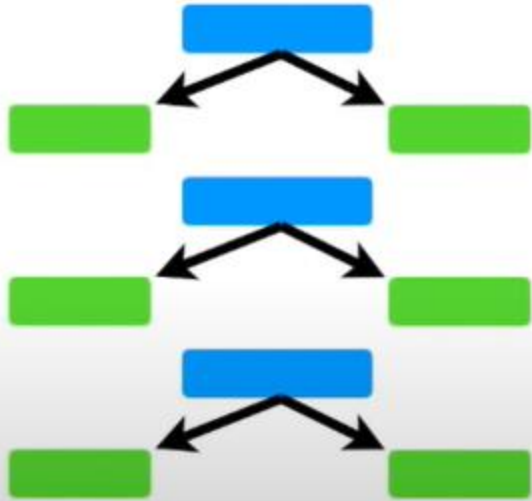


Amount of Say

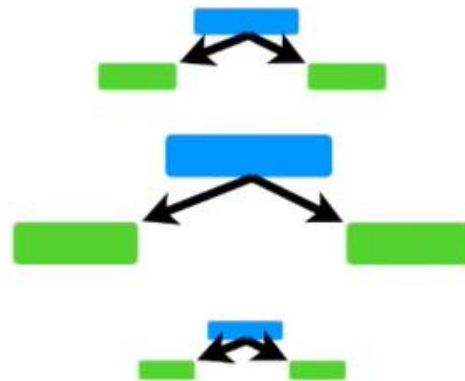


To review, the three ideas behind **AdaBoost** are...

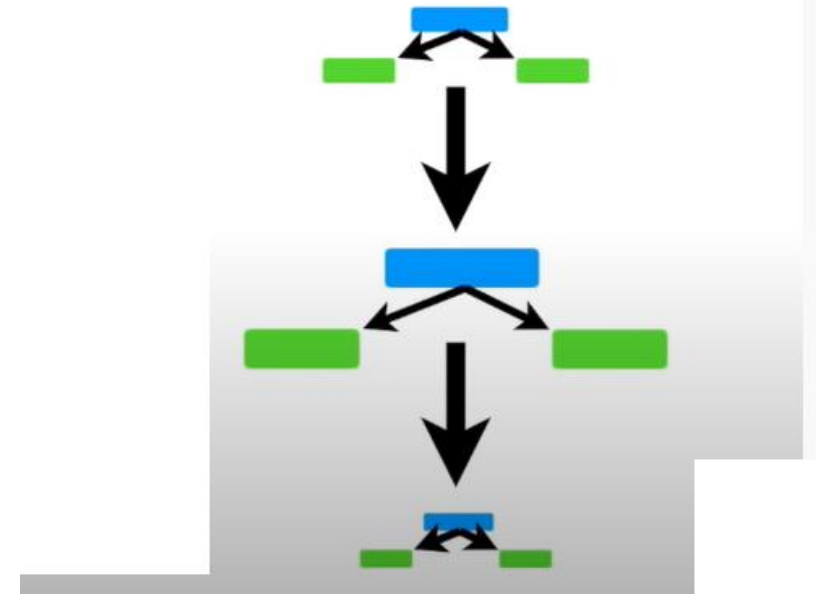
1) **AdaBoost** combines a lot of “weak learners” to make classifications. The weak learners are almost always **stumps**.



2) Some **stumps** get more say in the classification than others.



3) Each **stump** is made by taking the previous **stump's** mistakes into account.



If we have a **Weighted Gini Function**, then we use it with the **Sample Weights**, otherwise we use the **Sample Weights** to make a new dataset that reflects those weights.

3) Each **stump** is made by taking the previous **stump's** mistakes into account.

